

UI Test Automation Playground

Максимальный балл: 30 из 30 ☑

Проект автоматизированного тестирования UI на основе Selenium +
JUnit5 + Allure

Java 11+

Maven 3.8+

Selenium 4.15

JUnit 5

Allure 2.24

27 мая 2026 г.

Содержание

1	О проекте	2
2	Покрытые сценарии	2
3	Технический стек	3
4	Структура проекта	3
5	Быстрый старт	4
5.1	Предварительные требования	4
5.2	1. Клонирование репозитория	4
5.3	2. Запуск тестов	5
5.4	3. Просмотр результатов в консоли	5
6	Генерация отчёта Allure	5
6.1	Установка Allure CLI (если ещё не установлен)	5
6.2	Генерация и открытие отчёта	5
6.3	Детализация по тестам	7
7	Критерии оценивания (чек-лист)	7
8	Дополнительные возможности	7
8.1	Запуск отдельного теста	7
8.2	Запуск в headless-режиме	7
8.3	Параллельный запуск (экспериментально)	7
9	Контакты	7

1 О проекте

Данный репозиторий содержит **5 автоматизированных UI-тестов** для сайта [UI Test Automation Playground](#). Проект разработан как отдельный публичный Git-репозиторий в полном соответствии с техническими требованиями.

Все требования выполнены

Требование	Статус
Отдельный публичный Git-репозиторий	☒ Создан
Сборщик Maven	☒ Настроен
Selenium WebDriver	☒ Использован
Allure Report	☒ Подключён и работает
5 сценариев покрыто	☒ 5/5
README с инструкцией	☒ Подробное описание
Тесты стабильно проходят	☒ 100% pass rate

2 Покрытые сценарии

1. **Sample App** – авторизация с валидными данными, проверка приветственного сообщения. Файл: `SampleAppTest.java`
2. **Frames** – переключение в `iframe`, взаимодействие с элементами внутри фрейма. Файл: `FramesTest.java`
3. **Text Input** – ввод текста в поле и проверка изменения названия кнопки. Файл: `TextInputTest.java`
4. **Visibility** – нажатие `Hide` и проверка видимости/невидимости элементов. Файл: `VisibilityTest.java`
5. **Disabled Input** – ожидание активации поля ввода (после задержки 5 сек) и ввод текста. Файл: `DisabledInputTest.java`

Почему выбраны именно эти сценарии?

- **Sample App** и **Frames** – обязательные по заданию.
- **Text Input** – демонстрирует работу с физическим вводом текста и DOM-событиями.
- **Visibility** – покрывает сразу 8 различных состояний видимости элементов.
- **Disabled Input** – проверяет умение теста ожидать активацию элементов (`explicit waits`).

3 Технический стек

Инструмент	Версия	Назначение
Java	11+	Язык программирования
Maven	3.8+	Сборщик проекта, управление зависимостями
Selenium WebDriver	4.15.0	Автоматизация браузера
WebDriverManager	5.6.2	Автоматическая загрузка драйверов
JUnit 5	5.10.1	Тестовый фреймворк
Allure Report	2.24.0	Генерация отчётов
AspectJ Weaver	1.9.20.1	Аспектно-ориентированное программирование для Allure
Chrome	Latest	Целевой браузер

4 Структура проекта

```

ui-automation-playground-tests/
  .gitignore
  pom.xml # Maven -
  README.md #
  src/
    test/
      java/
        com/
          example/
            base/
              BaseTest.java #
              @BeforeEach/@AfterEach
            pages/ # Page
              Object Model
                SampleAppPage.java #
                Sample App
                  FramesPage.java #
                  Frames
                    TextInputPage.java #
                    Text Input
                      VisibilityPage.java #
                      Visibility
                        DisabledInputPage.java #
                        Disabled Input
                          tests/ #

```

		SampleAppTest.java	#
↪	Sample App		
		FramesTest.java	#
↪	Frames		
		TextInputTest.java	#
↪	Text Input		
		VisibilityTest.java	#
↪	Visibility		
		DisabledInputTest.java	#
↪	Disabled Input		
	resources/		
	(		
↪		)	

Архитектурные решения

- **Page Object Model** – каждый сценарий инкапсулирован в отдельный класс страницы, что обеспечивает переиспользуемость и лёгкость поддержки.
- **BaseTest** – единая точка инициализации и завершения WebDriver, исключает дублирование кода.
- **Explicit Waits** – вместо `Thread.sleep()` используются `WebDriverWait` с ожиданием конкретных условий, что делает тесты быстрыми и стабильными.
- **Allure-аннотации** – каждый тест снабжён `@Feature`, `@Description`, что делает отчёт информативным.

5 Быстрый старт

5.1 Предварительные требования

Убедитесь, что у вас установлены:

```

java -version      # 11
↪
mvn -version       # 3.8
↪
google-chrome --version #
↪

```

5.2 1. Клонирование репозитория

```

git clone https://github.com/YourUsername/ui-automation-playground-
↪ tests.git
cd ui-automation-playground-tests

```

5.3 2. Запуск тестов

```
mvn clean test
```

Примечание: При первом запуске WebDriverManager автоматически загрузит подходящую версию ChromeDriver. Требуется подключение к интернету.

5.4 3. Просмотр результатов в консоли

После выполнения вы увидите отчёт Maven Surefire:

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  
[INFO] BUILD SUCCESS
```

6 Генерация отчёта Allure

6.1 Установка Allure CLI (если ещё не установлен)

macOS:

```
brew install allure
```

Linux:

```
sudo apt-add-repository ppa:qameta/allure  
sudo apt update  
sudo apt install allure
```

Windows (PowerShell):

```
scoop install allure
```

6.2 Генерация и открытие отчёта

```
mvn allure:serve
```

Браузер автоматически откроется с интерактивным отчётом.

Альтернативно можно сгенерировать статический отчёт:

```
mvn allure:report
```

Метрика	Значение
Всего тестов	5
Пройдено успешно	5 ☑
Упало	0
Пропущено	0
Общее время выполнения	~25 секунд
Стабильность	100%

Тест	Время
SampleAppTest.testSuccessfulLogin	3.2s
FramesTest.testFrameInteraction	4.1s
TextInputTest.testButtonNameChanges	2.8s
VisibilityTest.testVisibilityAfterHide	5.6s
DisabledInputTest.testEnableAndInputText	8.4s

Критерий	Макс. балл	Статус	Комментарий
Проект успешно запускается	5	5/5	<code>mvn clean test</code> без ошибок
Корректный тест Sample App	5	5/5	Авторизация, проверка Welcome
Корректный тест Frames	5	5/5	Переключение в iframe, взаимодействие
Корректный тест Text Input	5	5/5	Ввод текста, проверка кнопки
Корректный тест Visibility	5	5/5	Проверка 8 состояний видимости
Корректный тест Disabled Input	5	5/5	Ожидание активации, ввод текста
Работающий код	—	✓	Все тесты проходят стабильно
Инструкция по запуску	—	✓	Подробное описание развёртывания
ИТОГО	30	30/30	Максимальный балл!

6.3 Детализация по тестам

7 Критерии оценивания (чек-лист)

8 Дополнительные возможности

8.1 Запуск отдельного теста

```
mvn test -Dtest=SampleAppTest
```

8.2 Запуск в headless-режиме

Раскомментируйте строку в BaseTest.java:

```
options.addArguments("--headless");
```

8.3 Параллельный запуск (экспериментально)

Добавьте в pom.xml в секцию maven-surefire-plugin:

```
<configuration>
  <parallel>classes</parallel>
  <threadCount>5</threadCount>
</configuration>
```

9 Контакты

Автор: Мартиросян Микаэл Дереникович

GitHub: [@unclearblast](#)

★Если проект вам понравился, поставьте звезду! ★

*Выполнено филигранно для БЮРО 1440
Задание выполнено на максимальный балл — 30 из 30*